

SugamFlow — Platform Architecture

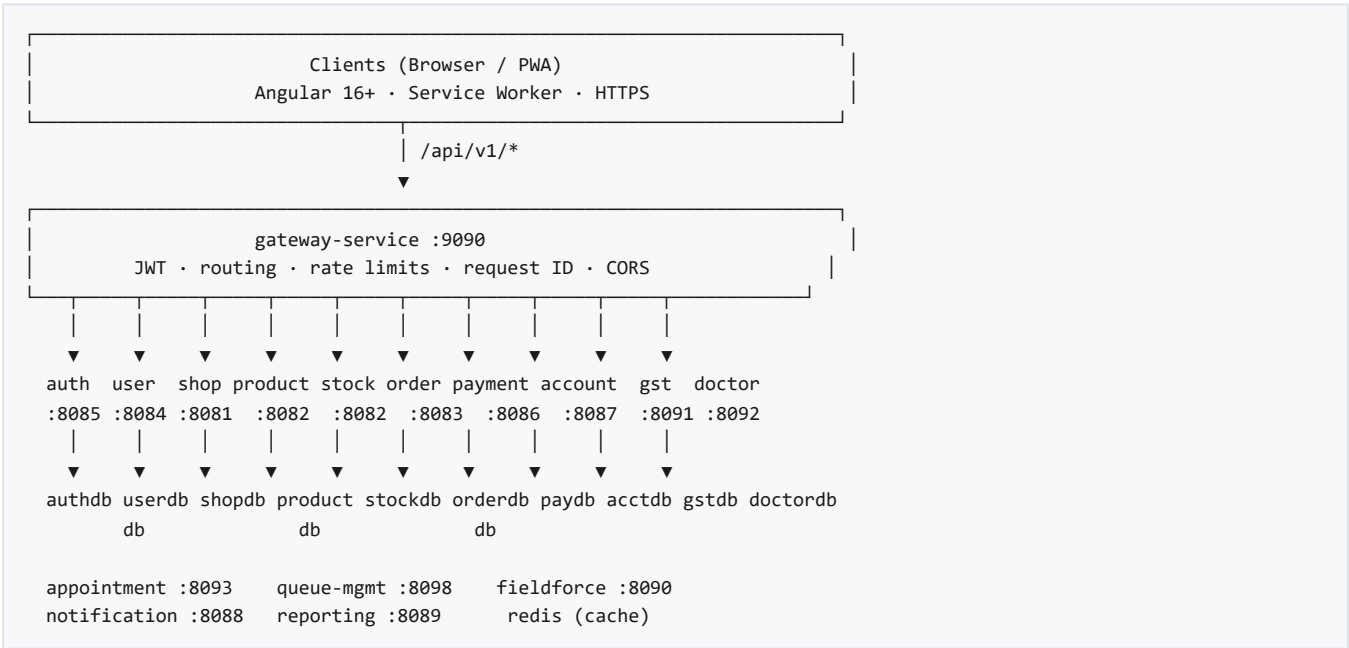
Platform: SugamFlow ERP
Document version: 1.0
Last updated: June 2026

1. Architecture style

SugamFlow uses a **microservices architecture** with:

- **Single-page application (Angular)** — shop-management-ui (port 4200 dev)
- **API Gateway** — gateway-service (port 9090) — routing, JWT validation, CORS
- **Service discovery** — Netflix Eureka (discovery-service)
- **Central config** — Spring Cloud Config (config-service)
- **Database-per-service** — PostgreSQL schemas/databases per microservice
- **Multi-tenancy** — tenantId + shopId on every request (X-Tenant-Id, X-Shop-Id headers)

2. High-level diagram



3. Microservices inventory

Service	Port	Database	Primary responsibility
gateway-service	9090	—	API edge, auth proxy, route to backends
discovery-service	8761	—	Eureka service registry
config-service	8888	—	Centralised Spring configuration
auth-service	8085	authdb	Login, JWT, password, shop-scoped users
user-service	8084	userdb	Customers / patients, UHID, registration
shop-service	8081	shopdb	Shops, tenants, subscriptions, modules
product-service	8082	productdb	Product catalog, import/export, medicines, lab tests
stock-service	8082	stockdb	Inventory, batches, expiry, procurement
order-service	8083	orderdb	Sales, billing, consultations, Rx, lab orders, reports
payment-service	8086	paymentdb	Payment records
account-service	8087	accountdb	Staff accounts, roles
gst-service	8091	gstdb	GST calculation, HSN/SAC, tax profiles
doctor-service	8092	doctordb	Doctor registry, clinical templates
appointment-service	8093	appointmentdb	Clinic appointments
queue-management-service	8098	queuedb	OPD queue tokens, reception board
fieldforce-service	8090	fieldforcedb	Salesman visits, leads, promoters
notification-service	8088	notificationdb	Email / SMS hooks
reporting-service	8089	reportingdb	Aggregated reports

4. Frontend architecture

Shell application

- **Path:** shop-management-ui/
- **Build:** Angular production bundles with lazy-loaded feature modules
- **PWA:** Service worker (ngsw-config.json) for offline shell caching

Feature modules (lazy loaded)

Module	Routes	Business types
workspace	Dashboard, orders, owner dashboard	All
healthcare	Doctor, reception, pharmacy, path-lab	Healthcare suite
inventory	Stock, procurement, PO	Retail, pharmacy, auto
admin	Platform admin, tenants	Super admin
public	Login, registration, landing	Public

Key cross-cutting UI services

- TenantContextService — active tenant, shop, branch
- LabelRuntimeService — Patient vs Customer vs Guest labels

- `capabilitiesForBusinessType()` — module visibility per shop type

5. API conventions

Pattern	Example
Gateway prefix	<code>http://host:9090/api/v1/...</code>
Auth	<code>Authorization: Bearer <JWT></code>
Tenant scope	<code>X-Tenant-Id: 100</code>
Shop scope	<code>X-Shop-Id: TRUST-POLY-01</code>
Role hint	<code>X-Auth-Role: SHOP_OWNER</code>
Tenant-wide clinical	<code>?tenantWide=true</code> on sales-admin APIs

6. Healthcare data flow (polyclinic network)

```
Reception (queue-service)
  → Doctor consult (order-service: consultations)
  → Prescription (order-service → pharmacy queue)
  → Lab order (order-service → path lab worklist)
  → Sample collection (path lab status updates)
  → Report version + PDF (order-service: lab_report_versions)
  → Lab bill (order-service → TRUST-PHAR or lab shop orders)
  → Patient chart (clinical-summary API, tenant-wide)
```

Shared tenant patient master: user-service customers with `tenantWide=true`.

Cross-outlet routing: `PolyclinicRoutingService` in order-service assigns `fulfillmentShopId` on prescriptions and routes lab orders to the tenant's path lab shop.

7. Data stores

Each service owns its PostgreSQL database (local dev: `host.docker.internal:5432`).

Database	Notable tables / domains
orderdb	orders, consultations, prescriptions, lab_orders, lab_report_versions
userdb	customers (patients), UHID
productdb	products, medicine_detail, lab test SKUs
stockdb	stock, batches, purchase_orders
shopdb	shops, tenants, enabled_modules
authdb	users, credentials (BCrypt)

Migrations: Flyway per service (v1...v25 on order-service). Local dev may use `ddl-auto=update` with Flyway disabled.

8. Security model

- **JWT** issued by auth-service; validated at gateway

- **Shop-scoped users:** username_SHOPID pattern (e.g. RajeevRanjan_TRUST-POLY-01)
- **Internal service calls:** SECURITY_INTERNAL_API_KEY header
- **Role permissions:** SHOP_OWNER, DOCTOR, RECEPTION, LAB_OWNER, SUPER_ADMIN, etc.
- **Production:** SPRING_PROFILES_ACTIVE=prod, ddl-auto=validate, secrets in .env.production

9. Deployment topology

Local development

```
Docker Compose (docker-compose.yml)
→ All microservices + Redis + MailHog
→ PostgreSQL on host (multiple databases)
→ start-local.ps1 / compose-local.ps1
→ UI: npm run start:clean (port 4200, proxy to gateway 9090)
```

Production (typical)

```
AWS EC2 / ECS
→ Docker images (IMAGE_PREFIX/IMAGE_TAG)
→ RDS PostgreSQL (multi-database or schemas)
→ Nginx or ALB → gateway-service:9090
→ Static UI (dist/) on CDN or Nginx
→ Optional: RabbitMQ profile for async events
```

10. Observability & operations

- **Health:** /actuator/health on each service
- **Request tracing:** X-Request-Id propagated gateway → services
- **Timezone:** Asia/Kolkata (IST) across JVM and UI
- **Scripts:** scripts/test-all-services.ps1, healthcare E2E, polyclinic full-flow

11. Technology stack

Layer	Technology
Frontend	Angular 16+, Bootstrap 5, RxJS, PWA
API Gateway	Spring Cloud Gateway
Backend	Spring Boot 3, Java 17+
ORM	JPA / Hibernate
DB	PostgreSQL 14+
Cache	Redis
Build	Maven (services), npm (UI)
Containers	Docker, Docker Compose

12. Scalability principles

1. **Add business types** via capability registry — no new deployable UI app.
 2. **Add outlets** under one tenant — shared patients and clinical records.
 3. **Horizontal scale** — stateless services behind gateway; DB connection pooling (Hikari).
 4. **Lazy UI chunks** — healthcare, inventory, admin load on demand (~500 kB chunks).
-

Repository root: sugamflow monorepo. UI: shop-management-ui. Infrastructure: docker-compose.yml, infra/postgres/.